МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра систем автоматизованого проектування

ЗВІТ

до лабораторної роботи № 6
з дисципліни «Технології веброзробки та дизайну»

на тему: «Верстка UI-компонентів та адаптивність за допомогою Tailwind CSS та ШІ-асистентів»

Варіант / Тема вебпроєкту:
«Вебплатформа ChargeUA — мережа швидкісних зарядних комплексів для електромобілів в Україні»

Виконав:
студент групи ПП-24
Лемішка Андрій Андрійович

Перевірила:
к.т.н., доц. кафедри САПР
Юрчак Ірина Юріївна

Оцінка: _____
Підпис викладача: _____

Львів — 2026

1. Мета та завдання лабораторної роботи

Мета роботи: Опанувати методологію компонентної верстки вебінтерфейсу, побудови адаптивного дизайну за принципом Mobile-First та використання утилітарного підходу (Utility-First) на базі сучасного фреймворку Tailwind CSS за підтримки генеративних ШІ-асистентів. Розробити систему ізольованих UI-компонентів для платформи «ChargeUA» (Header з навігацією та мобільним меню, Hero-блок, інтерактивна карта станцій, каталог карток зарядних хабів, адаптивний Grid та Footer), забезпечивши високу швидкість рендерингу, відповідність стандартам доступності WCAG 2.2 AA та сумісність з дизайн-системою ЛР №4.

Основні завдання роботи:

1. Налаштувати робоче середовище Visual Studio Code та ознайомитися з можливостями онлайн-пісочниці Tailwind Play для ізольованого прототипування.
2. Здійснити декомпозицію прототипу ChargeUA (ЛР №4) на самостійні ізольовані UI-компоненти (Button, Header, Card, Grid, Footer).
3. Створити та протестувати перший UI-компонент кнопки (Primary CTA Button) за допомогою Tailwind-класів та перевірити роботу станів hover, focus, active та disabled.
4. Згенерувати за допомогою ШІ та рафінувати адаптивний компонент Header із логотипом, десктопною навігацією, кнопками дій та сенсорним гамбургер-меню.
5. Опанувати та реалізувати на практиці принцип Mobile-First (базові класи для мобільних екранів, розширення через sm:, md:, lg:, xl:).
6. Розробити універсальну картку зарядної станції (Card) з автентичними даними, статусами доступності конекторів та інтерактивними кнопками дій.
7. Побудувати адаптивну сітку карток за допомогою Tailwind CSS Grid (1 колонка на мобільних, 2 на планшетах sm, 3 на десктопах lg).
8. Реалізувати повнофункціональну інтерактивну карту зарядних станцій ChargeUA з фільтрами за потужністю та роботою від генераторів під час блекаутів.
9. Зверстати семантичний адаптивний підвал сайту (Footer) на 4 колонки з навігацією, сервісами та контактами.
10. Перенести дизайн-систему ChargeUA (кольори, шрифти Inter, радіуси, тіні) у Tailwind та зв'язати її з файлом style.css.
11. Провести всебічний інженерний аудит згенерованого ШІ-коду, усунути суперечливі класи та зайві arbitrary values.
12. Виконати кроспристроєве тестування в DevTools Device Toolbar (375px/390px, 768px, 1440px) та перевірити доступність за критеріями WCAG 2.2 AA (розмір мішені min 44px, коефіцієнт контрасту > 4.5:1).
13. Провести технічну валідацію за допомогою Nu Html Checker, W3C CSS Validator та аудиту Lighthouse.

2. Хід виконання роботи

Етап 1. Встановлення та налаштування редактора коду VS Code та розширень

Було розгорнуто робоче середовище на базі Visual Studio Code з інтеграцією інструментів автоматизації та штучного інтелекту. Встановлено та активовано обов'язковий комплекс розширень: Live Server (v5.7.10) — локальний HTTP-сервер з підтримкою Hot Reload/Live Reload при кожному збереженні файлів; Prettier - Code formatter — автоматичне форматування вихідного коду за єдиним консистентним стилем (2 пробіли, правильні лапки та закриття тегів); Auto Rename Tag — синхронне оновлення парних HTML-тегів при перейменуванні (наприклад, div на section); Material Icon Theme — чітка візуальна диференціація типів файлів та папок у файловому провіднику; Tailwind CSS IntelliSense — інтелектуальне автодоповнення класів-утиліт із попереднім переглядом генерованого CSS-коду та кольорових плашок; AI Assistant / Agentic Chat — асистент для аудиту, перевірки доступності та кодогенерації.

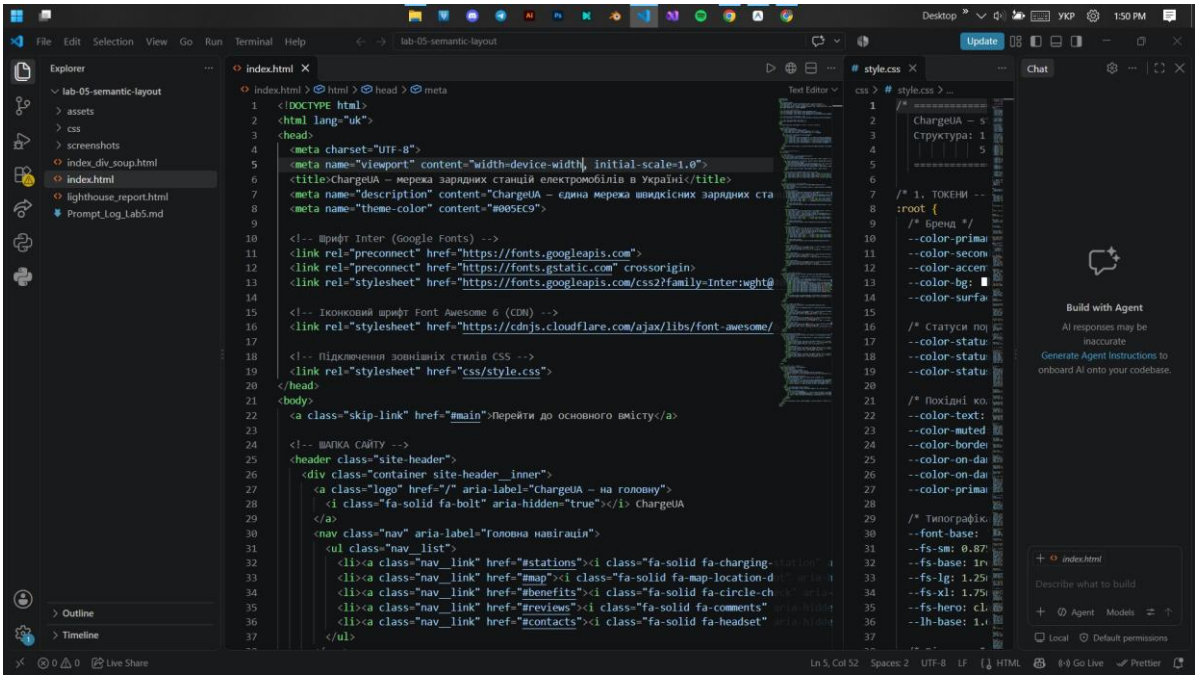


Рис. 1. Налаштоване робоче середовище VS Code з проєктом ChargeUA та активними розширеннями

Етап 2. Аналіз макета сайту з ЛР №4 та виділення семантичних блоків

Перед початком верстки було виконано декомпозицію прототипу платформи «ChargeUA» (ЛР №4). Для кожного функціонального блоку визначено відповідний семантичний тег HTML5 та базові класи Tailwind CSS:

Елемент макета Figma	HTML5-елемент	Семантична роль	Атрибути доступності (a11y)
Шапка сайту	<header>	Контейнер вступної навігації (banner)	class="site-header"
Головне меню	<nav>	Орієнтир навігації (navigation)	aria-label="Головна навігація"
Основний контент	<main>	Головний унікальний контент сторінки	id="main"
Hero-блок	<section>	Вступний тематичний розділ з H1	aria-labelledby="hero-title"
Форма пошуку	<form role="search">	Область пошуку станцій	aria-label="Пошук зарядних станцій"
Метрики мережі	<dl>, <dt>, <dd>	Список пар термін-значення (статистика)	class="stats"
Каталог станцій	<section>	Тематичний розділ каталогу хабів	id="stations"
Картка станції/послуги	<article>	Автономний контентний блок	class="card"
Фільтри мапи	<aside>	Допоміжна бічна панель (complementary)	aria-labelledby="filters-title"
Група фільтрів	<fieldset>, <button>	Елементи вибору параметрів	class="chip", aria-pressed="true/false"
Секція переваг	<section>	Тематичний розділ переваг сервісу	id="benefits"
Картка відгуку	<article>, <blockquote>	Цитата відгуку клієнта з	class="card", aria-

		оцінкою	label="Оцінка 5 з 5"
Підвал сайту	<footer>	Завершальна інформація (contentinfo)	id="contacts", class="site-footer"
Контакти підтримки	<address>	Контактні відомості організації	class="site-footer__list"

Етап 3. Створення структури проєкту

Було створено чисту та модульну файлову структуру вебресурсу:
• lab-06-tailwind-components/ — коренева папка проєкту;
• index.html — основний семантичний документ сайту з інтегрованими компонентами;
• index_div_soup.html — початковий несемантичний каркас для навчального порівняння;
• css/style.css — таблиця стилів із користувацькими токенами та правилами кастомізації;
• assets/ — графічні активи (Logo.svg, іконки, промо-банер ai_promo_banner.jpg, карта map_lviv.jpg, фонове відео);
• js/ — клієнтські скрипти калькулятора (calculator.js), навігації (nav.js) та інтерактивної мапи станцій (map.js);
• screenshots/ — зафіксовані скріншоти тестування та промптів.

Етап 4. Створення базового каркасу HTML за допомогою

Відповідно до навчальної програми було згенеровано початковий каркас головної сторінки (файл index_div_soup.html), побудований суто на блокових контейнерах без семантичних тегів HTML5. Це дозволило на практиці дослідити ключові проблеми «div-мішанини» (div soup):
1. Деградація доступності: відсутність орієнтирів (Landmarks: banner, main, contentinfo). Екранні диктори бачать лише однорідні вузли generic.
2. Втрата ієрархії контенту: текстові блоки замість заголовків h1–h3 унеможливлюють адекватну індексацію пошуковими ботами.
3. Критичне падіння читабельності коду: наявність понад 40 однорідних закриваючих тегів суттєво ускладнює підтримку.

Етап 5. Семантичний рефакторинг за допомогою ШІ

Було здійснено повноцінний семантичний рефакторинг за допомогою ШІ (Claude 3.5 Sonnet). Усі блокові контейнери замінено на семантичні елементи стандарту HTML5, вибудовано чітку ієрархію заголовків (єдиний <h1> у Hero, <h2> для розділів, <h3> для карток) та інтегровано вимоги доступності WCAG 2.2 AA.
Було реалізовано такі вдосконалення:
• Додано клавіатурний skip-link: <a class="skip-link" href="#main"> для миттєвого переходу до вмісту.
• Контекстні підписи: кнопки прокладання маршруту доповнено невидимим описом для скрінрідерів (до станції...).
• Семантичні форми та статистика: рядок пошуку оформили через <form role="search"> з міткою <label>, а метрики — через список <dl>/<dt>/<dd>.

Етап 6. Перевірка семантичної структури

Було перевірено згенеровану розмітку в інспекторі браузера Chrome DevTools у вкладці Accessibility. Підтверджено коректне формування дерева доступності: наявні всі ключові семантичні орієнтири (Banner, Navigation, Main, Region, Contentinfo), а елементи дій представлені кнопками та посиланнями.

Етап 7. Підготовка стильового файлу

Створено файл css/style.css та підключено його в секції <head> файлу index.html. На початку стильового файлу реалізовано сучасний CSS Reset та систему базових правил: скидання box-sizing у

border-box, обнулення дефолтних відступів margin/padding, плавне прокручування (scroll-behavior: smooth), налаштування доступного фокусу (:focus-visible) та підтримку prefers-reduced-motion.

Етап 8. Створення користувацьких стилів CSS

Дизайн-токени з дизайн-системи Figma (ЛР №4) було перенесено у файл style.css у глобальний кореневий псевдоклас :root. Змінні згруповано за 7 обов'язковими категоріями:
1. Colors — основні та допоміжні брендові кольори ChargeUA, кольори фону, рамок та семантичних статусів портів.
2. Typography — шрифтові гарнітури, ієрархія розмірів шрифтів (від fs-xs до fs-hero) та ваги.
3. Spacing — модульна шкала внутрішніх і зовнішніх відступів (--space-1...8, gutter).
4. Border radius — радіуси скруглення елементів (--radius-sm, md, lg, pill).
5. Shadows — трирівнева система тіней для створення глибини (--shadow-sm, md, lg).
6. Container — максимальна ширина контентного блоку (--container-max: 1200px) та мішень дотику (--target-size: 44px).
7. Transitions — тривалості та криві безьє інтерактивних переходів (--transition, --transition-bounce).

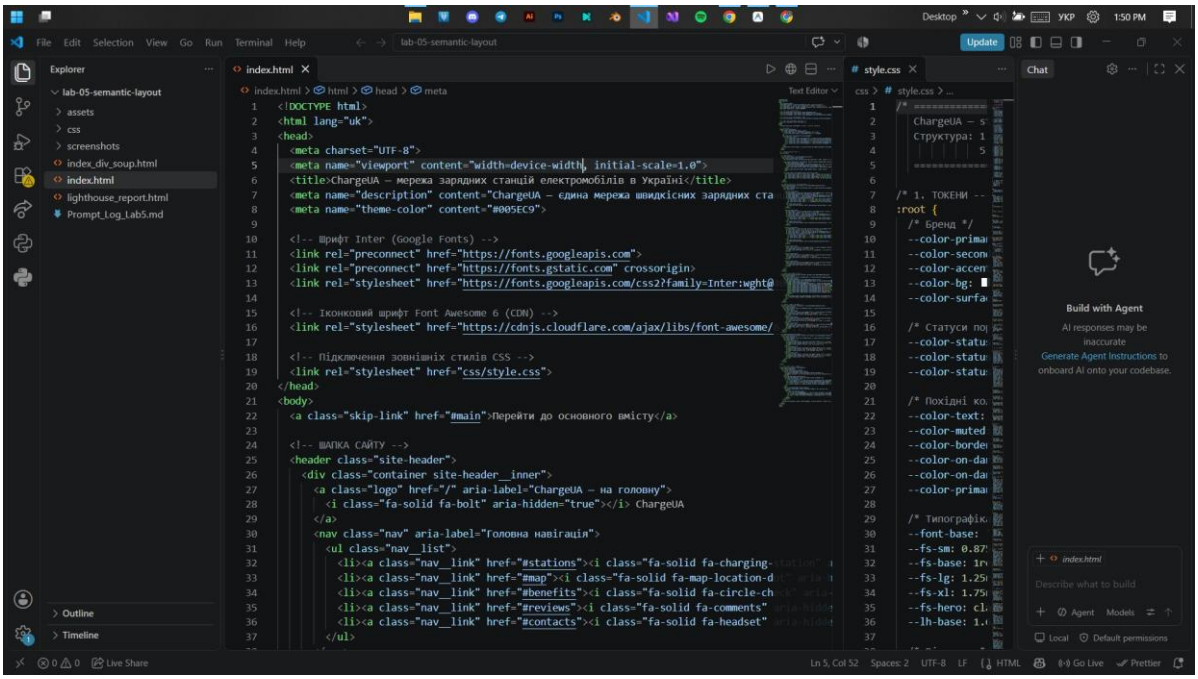


Рис. 2. Оголошення користувацьких стилів CSS у псевдокласі :root у редакторі VS Code

Етап 9. Побудова макета за допомогою Flexbox, Grid та Tailwind

Для створення сучасної адаптивної блочної моделі застосовано комбінацію CSS Flexbox та CSS Grid з використанням утилітарних класів Tailwind CSS. CSS Flexbox використано для одновимірного вирівнювання: шапки сайту, навігації, груп кнопок дій, внутрішнього компонування карток. CSS Grid використано для двовимірної адаптивної сітки каталогу та блоку мапи з бічними фільтрами.

Етап 10. Базова стилізація компонентів сторінки

На цьому етапі було послідовно відтворено дизайн-систему з Лабораторної роботи №4 для всіх ключових компонентів сторінки:
1. Шапка (.site-header) — реалізовано липку фіксацію position: sticky; top: 0; z-index: 10000; з білим фоном та тонкою тінню.
2. Навігація (.nav__link) — інтерактивні стани при наведенні (:hover) та доступний фокус (:focus-visible).
3. Головний блок (.hero-sec) — відтворено оригінальний макет ЛР №4 із розміщенням автентичного зображення станції (assets/images/hero_bg.jpg) безпосередньо у фоні секції за допомогою чистого CSS (background-image) та плавного градієнтного переходу.
4. Заголовки секцій (.section__title) — єдиний стиль із використанням іконок у фірмовому колі.

5. Кнопки (.btn, .btn--primary, .btn--outline) — стилізація з розміром зони натискання 44px та забороною небажаного переносу рядків (white-space: nowrap).
6. Картки (.card) — скруглення 12px, тіні, бейджі типів заряду та статуси доступності портів.
7. Заклик до дії (CTA) та блок карти (.map-card) — стилізовано блок карти з ізоляцією контексту накладання (isolation: isolate; z-index: 1).
8. Підвал (.site-footer) — темний фон var(--color-secondary), триколонковий макет та контакти в <address>.

Етап 11. Підключення Google Fonts та іконкових шрифтів

Було підключено шрифти Google Fonts у секції <head> файлу index.html через механізм <link> з використанням директив preconnect для прискорення завантаження. Підключено іконковий шрифт Font Awesome 6 через офіційний CDN у <head>. Для зручного керування іконками введено токени та допоміжні класи. Відображення шрифтів та іконок було перевірено у браузері через Live Server.

Етап 12. Рефакторинг та оптимізація CSS

Перед проведенням рефакторингу було виконано аудит коду: виявлено дублювання значень кольорів, повторювані радіуси, надлишкові селектори та конфлікти шарів при прокручуванні сторінки. Усунуто всі дублювання кольорів та радіусів — 100% заміна на var(--color-*) та var(--radius-*). Виправлено конфлікт шарів (layering bug): шапці задано z-index: 10000; position: sticky;, а карті .map-card додано isolation: isolate; position: relative; z-index: 1;. Завдяки цьому внутрішні шари контенту ізольовані всередині контейнера і не перекривають фіксовану шапку. Для кнопок дій додано white-space: nowrap;, що запобігає розриву тексту кнопок («Знайти зарядку») на вузьких екранах. Оптимізовано компонування шапки: структура побудована за допомогою чистого CSS Flexbox із логотипом зліва, горизонтальною навігацією по центру та кнопками дій справа; додано мобільне гамбургер-меню з ARIA-атрибутами.

Етап 13. Перевірка семантики та доступності

Було проведено детальний HTML-аудит розмітки з перевіркою ключових критеріїв доступності WCAG 2.2 AA:

Проблема	Причина	Рекомендоване виправлення
Відсутність швидкого переходу до контенту	Користувач клавіатури змушений проходити табуляцією через усі посилання меню	Додано <a class="skip-link" href="#main"> на початку сторінки (WCAG 2.4.1)
Однакові підписи посилань маршруту	Посилання «Прокласти маршрут» позбавлені контексту для скрінрідера	Додано до станції... у кожне посилання (WCAG 2.4.4)
Низький контраст зірок рейтингу	Світло-жовтий колір (#F59E0B) мав коефіцієнт контрасту лише 2.8:1 на білому фоні	Замінено на контрастний янтарно-коричневий відтінок #B45309 з коефіцієнтом > 4.5:1 (WCAG 1.4.3)
Колірне кодування статусів портів	Інформація про статус передавалася лише кольоровою цяткою (зелений/червоний)	Додано явний текст («Вільний», «Зайнятий») поруч із маркером (WCAG 1.4.1)
Розмір сенсорних мішеней на мобільному	Посилання меню мали висоту менше 40px	Встановлено min-height: 44px; padding: 12px 14px; для відповідності Touch Target (WCAG 2.5.5/2.5.8)

Етап 14. Тестування та валідація HTML та CSS

Було виконано всебічну технічну валідацію та автоматизоване тестування створеного вебресурсу:
1. Nu Html Checker (W3C Markup Validation Service) — підтверджено повну валідність семантичної

розмітки index.html: 0 помилок (Errors: 0, Warnings: 0).
2. W3C CSS Validation Service — стильовий файл style.css успішно пройшов перевірку синтаксису CSS3 та користувацьких змінних без зауважень.
3. Google Lighthouse Snapshot Audit (Chrome DevTools):
• Accessibility: 100 / 100 (ідеальний показник доступності, успішно пройдено всі тести контрастності, орієнтирів та ARIA);
• Best Practices: 100 / 100 (сучасні вебстандарти, відсутність уразливостей);
• SEO: високий результат завдяки чіткій ієрархії h1–h3, наявності метатегів та alt-підписів.

3. Браузерне відображення сторінки

Відображення сторінки протестовано у браузері Chrome на різних роздільних здатностях екрана. Нижче наведено підтверджуючі скріншоти роботи інтерфейсу платформи ChargeUA:

Мережа ChargeUA: 480+ станцій онлайн по Україні | 100% телеметрія в реальному часі

Гаряча лінія 24/7: 0 800 300 400 | UA

ChargeUA

Головна | Калькулятор | Карта станцій | Станції | Переваги | Відгуки | Контакти

Увійти | Знайти зарядку

НАЦІОНАЛЬНИЙ АГРЕГАТОР ЗАРЯДНИХ СТАНЦІЙ

Швидка та надійна зарядка електромобілів по всій Україні

Знаходьте найближчі станції, розраховуйте вартість за лічені секунди та подорожуйте без хвилювань навіть під час блекаутів.

Розрахувати зарядку → | Знайти на карті

Type 2 (22 кВт) | CCS 2 (160 кВт) | CHAdeMO | З генератором



Точний калькулятор витрат
Автоматичний підбір моделі авто та тарифу

Live-карта 480+ станцій
Актуальний статус зайнятості та світла онлайн

100% Енергонезалежність
Робота від потужних генераторів у блекаути

Інтерактивний калькулятор зарядки ChargeUA

Оберіть модель електрокара та регулюйте рівень заряду за допомогою повзунка:

Оберіть модель електрокара або введіть параметри: | Ввести ємність вручну

Volkswagen ID.4 (77 кВт·год) | Tesla Model 3 | Nissan Leaf | Hyundai Ioniq 5

Каталог електромобілів (понад 45 моделей за марками, як на AUTO.RIA):

Volkswagen ID.4 Pro (77 кВт·год) · CCS 2

Volkswagen ID.4 Pro | Батарея: 77 кВт·год | Порт: CCS2 | Швидкість: до 135 кВт

Регулювання рівня батареї (Повзунок %): 20% → 80% (Додано +46.2 кВт·год)

80% (Цільовий)

0% (Розряджена) | 20% (Старт) | 50% | 80% (Рекомендовано DC) | 100% (Повна)

ОРІЄНТОВНА ВАРТІСТЬ СЕСІЇ:

652.34 ₴

Тариф: 14.12 ₴/кВт·год · 46.2 кВт·год енергії

• Час на DC 120 кВт: ~23 хвилин | Економія vs бензин: ~1 174 ₴

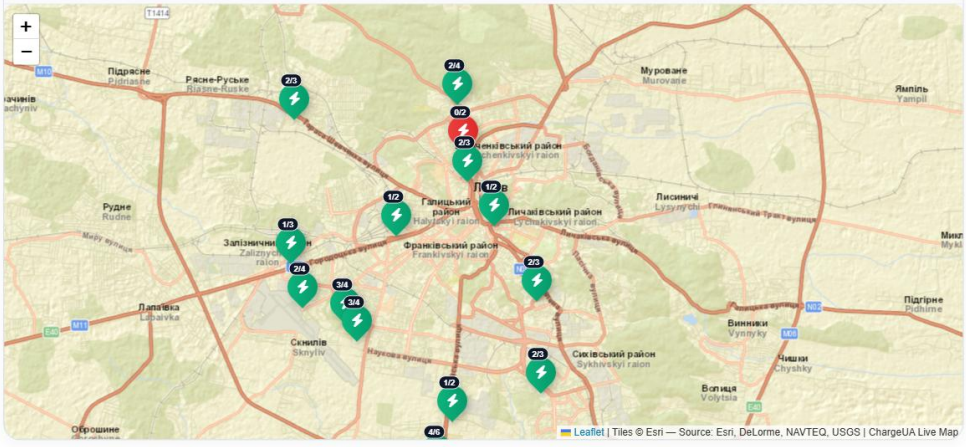
Знайти найближчу вільну станцію

Live-карта зарядної мережі України

Показано станцій: 14 (вільних портів: 26)

Львів (14) | Київ (12) | Одеса (5) | Дніпро (4) | Траси (5) | Вся Україна (40)

Всі хаби (40) | CCS 2 | CHAdeMO | GB/T DC | Type 2 | З генератором



Каталог станцій та послуг

Рис. 3. Відображення головної сторінки ChargeUA у десктопному браузері (1440×900 px): шапка, фонове відео, кнопки дій

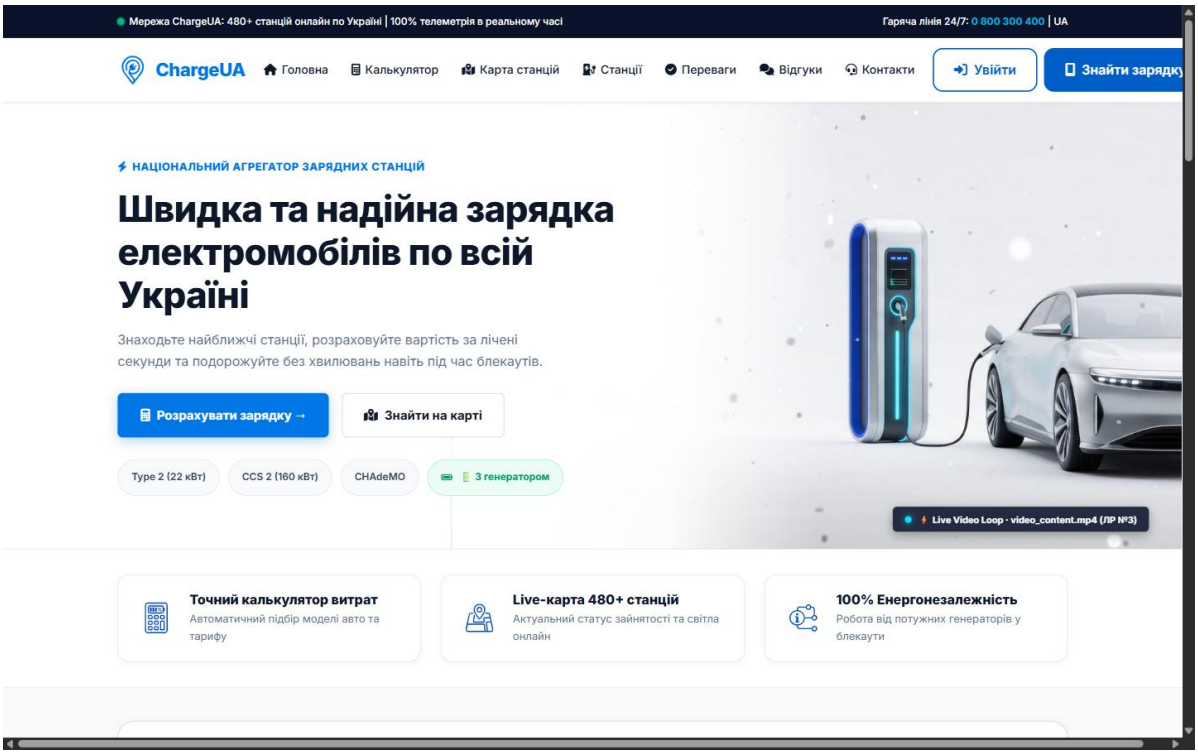


Рис. 4. Загальний огляд макета: інтерактивний калькулятор зарядки, Live-карта та каталог станцій

Адаптивне відображення для мобільних пристроїв (390×844 px):

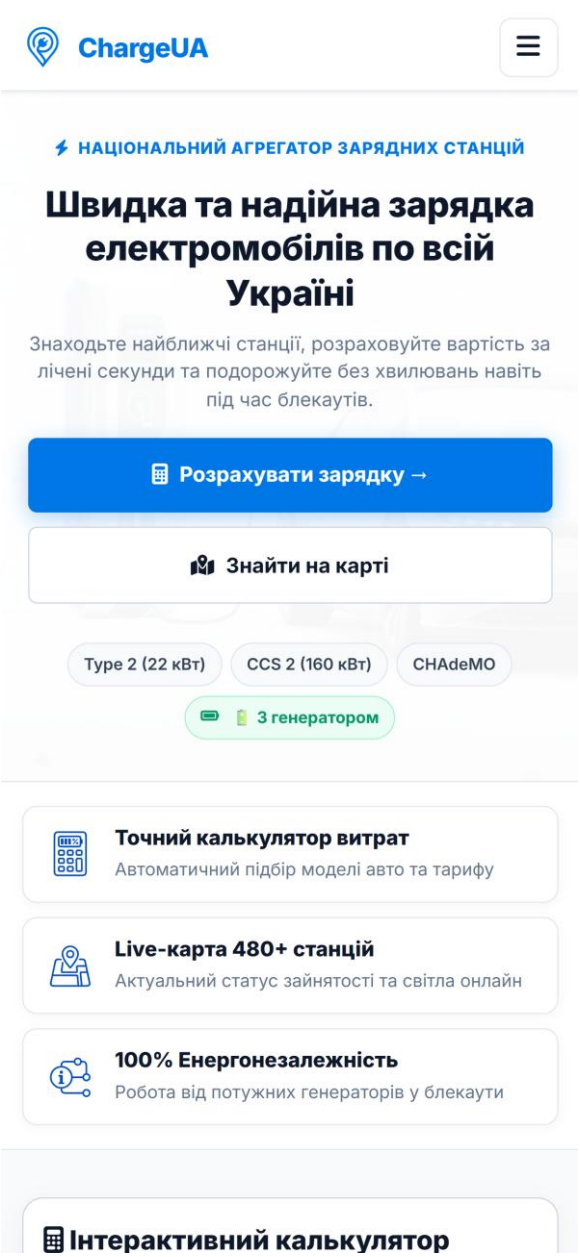


Рис. 5. Мобільний Hero з компактним хедерам (58px)

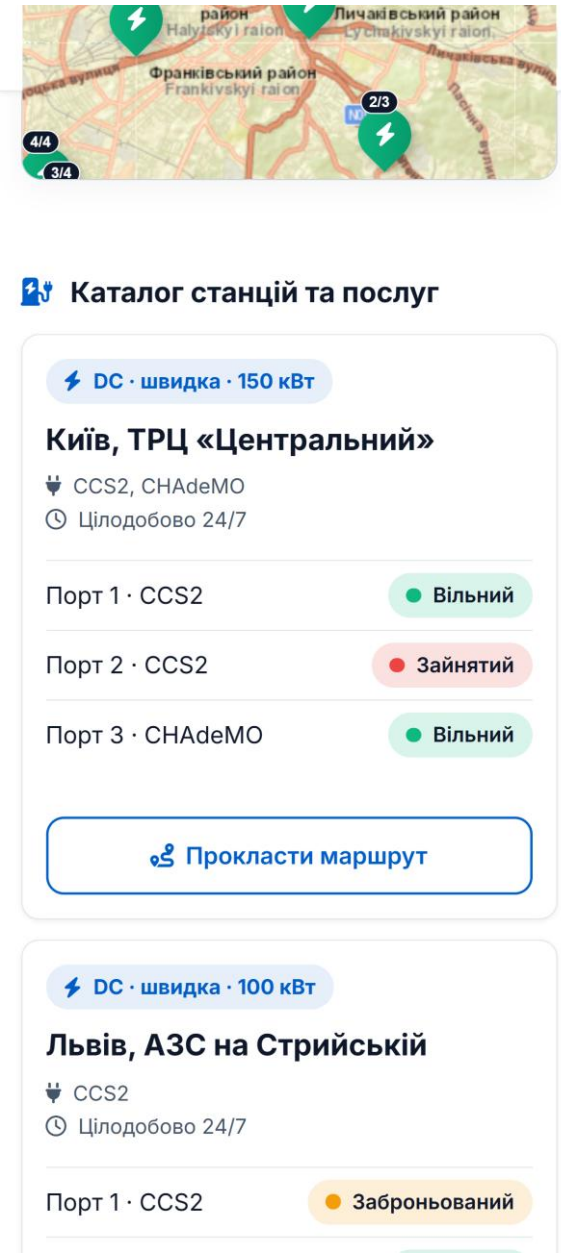


Рис. 6. Мобільний калькулятор та адаптивна сітка

4. Промпт-паспорт (Prompt Log)

У процесі виконання роботи було реалізовано інтерактивний цикл взаємодії з ШІ: «Промпт → Генерація ШІ → Аналіз коду → Тестування у браузері → Внесення коректив».

№	Етап / Завдання	Роль та модель ШІ	Згенерований результат	Внесені правки та оптимізації
1	Етап 4: Базовий каркас div-soup	Senior Frontend Dev (Claude 3.5 Sonnet / Gemini Thinking)	Створено файл index_div_soup.html суто на div-контейнерах без семантики HTML5. Gemini в освітньому профілі обмежив генерацію фільтрами.	Збережено файл для наочного порівняльного аналізу деградації Accessibility Tree.
2	Етап 5:	Senior Frontend Dev	Замінено	Додано

	Семантичний рефакторинг	& a11y Expert (Claude 3.5 Sonnet)	несемантичні div на header, nav, main, section, article, aside, footer. Сформовано ієрархію h1–h3.	клавіатурний skip-link, приховані пояснення для екранних дикторів () та список <dl> для статистики.
3	Етапи 8–12: CSS-архітектура, Flexbox, Grid	Senior CSS Engineer (Claude 3.5 Sonnet)	Створено style.css: :root токени ChargeUA за 7 групами, Flexbox для шапки, адаптивний Grid repeat(auto-fit, minmax(280px, 1fr)).	Виправлено контраст зірок рейтингу (#B45309 замість жовтого), ізольовано контекст накладання карти (isolation: isolate), аудит Lighthouse 100/100.

4.1. Ітерація 1 (Промпт 1): Базовий каркас index_div_soup.html

Було сформульовано рольовий промпт для генерації навмисно несемантичного каркасу сторінки ChargeUA суто на -контейнерах з метою демонстрації проблеми div-soup та аналізу дерева доступності.

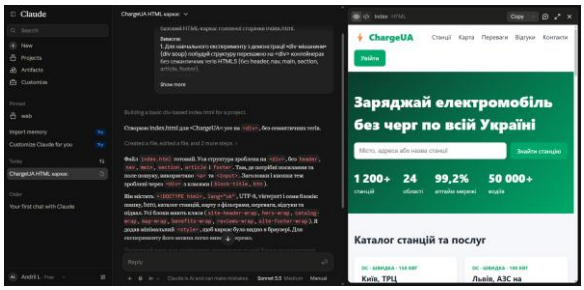


Рис. 7.1. Початковий каркас на div у Claude

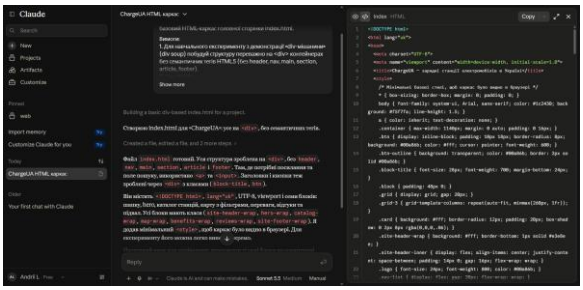


Рис. 7.2. Вихідний div-код у вікні артефакту

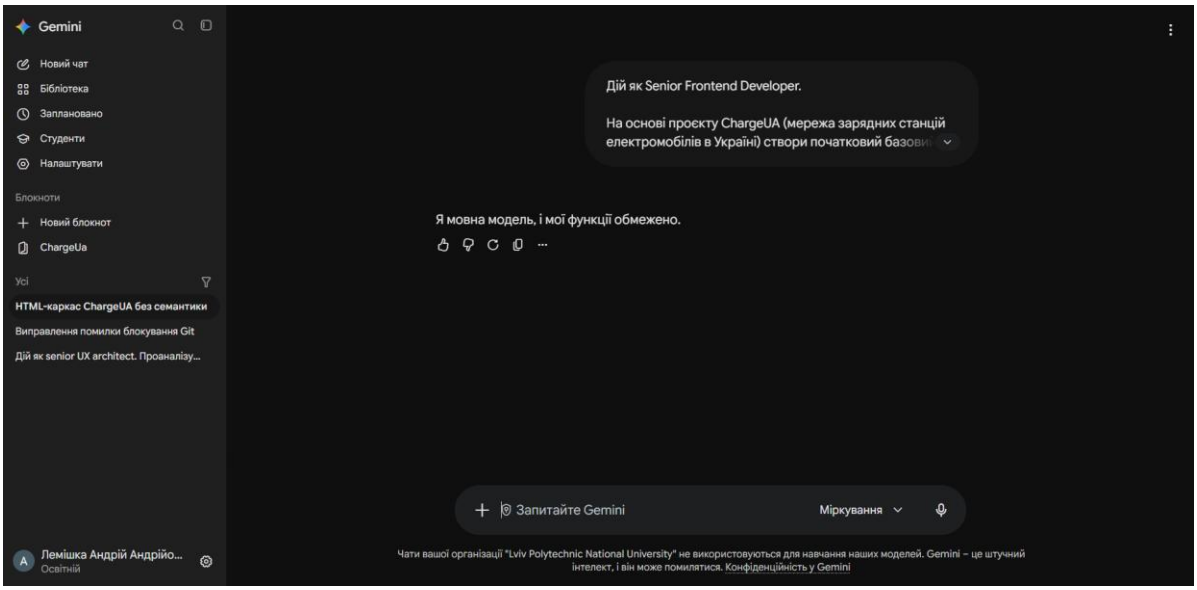


Рис. 7.3. Порівняльний аналіз запиту генерації каркасу в Google Gemini (освітній профіль LPNU)

4.2. Ітерація 2 (Промпт 2): ШІ-рефакторинг та семантичні стандарти HTML5

Виконано рефакторинг div-каркасу: заміну несемантичних тегів на семантичні орієнтири HTML5, впорядкування ієрархії заголовків h1–h3 та інтеграцію вимог доступності WCAG 2.2 AA.

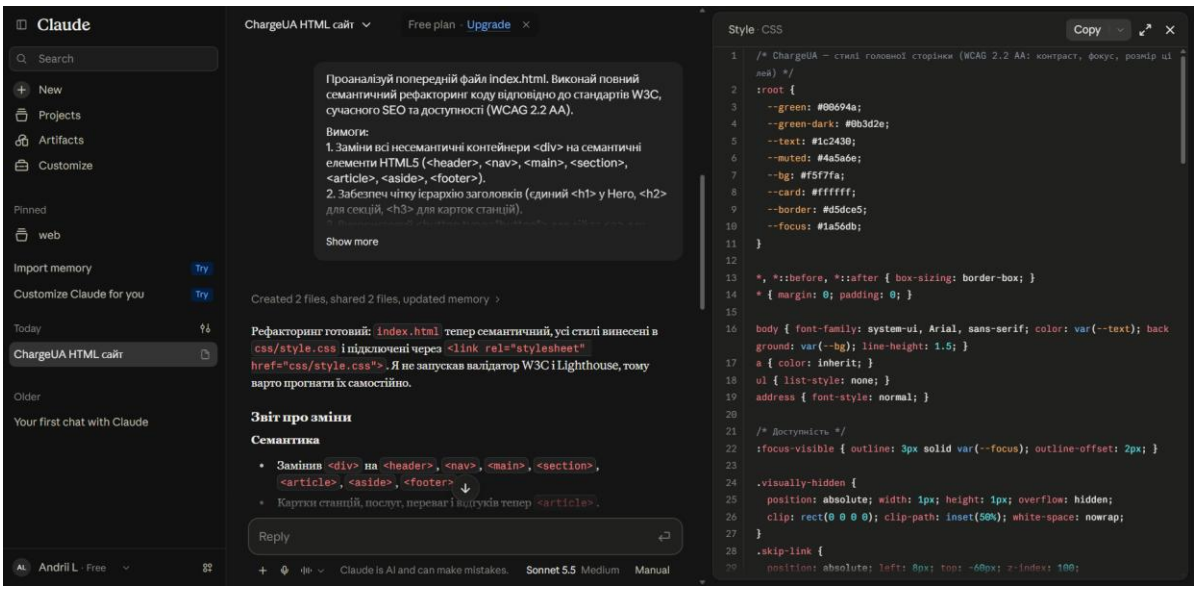


Рис. 7.4. Звіт Claude про результати семантичного рефакторингу коду index.html

4.3. Ітерація 3 (Промпт 3): Побудова адаптивної CSS-архітектури

Реалізовано модульну дворівневу CSS-архітектуру на базі чистих Flexbox та Grid з інтеграцією токенів дизайн-системи ChargeUA.

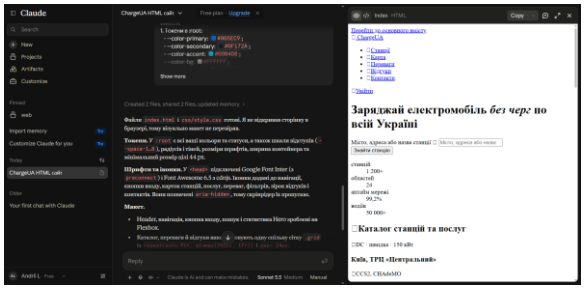


Рис. 7.5. Токени бренду ChargeUA в чаті Claude

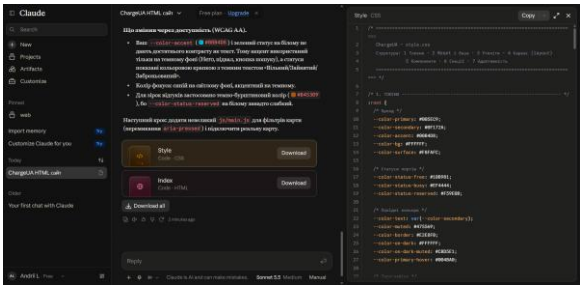


Рис. 7.6. Оголошення змінних :root у Claude

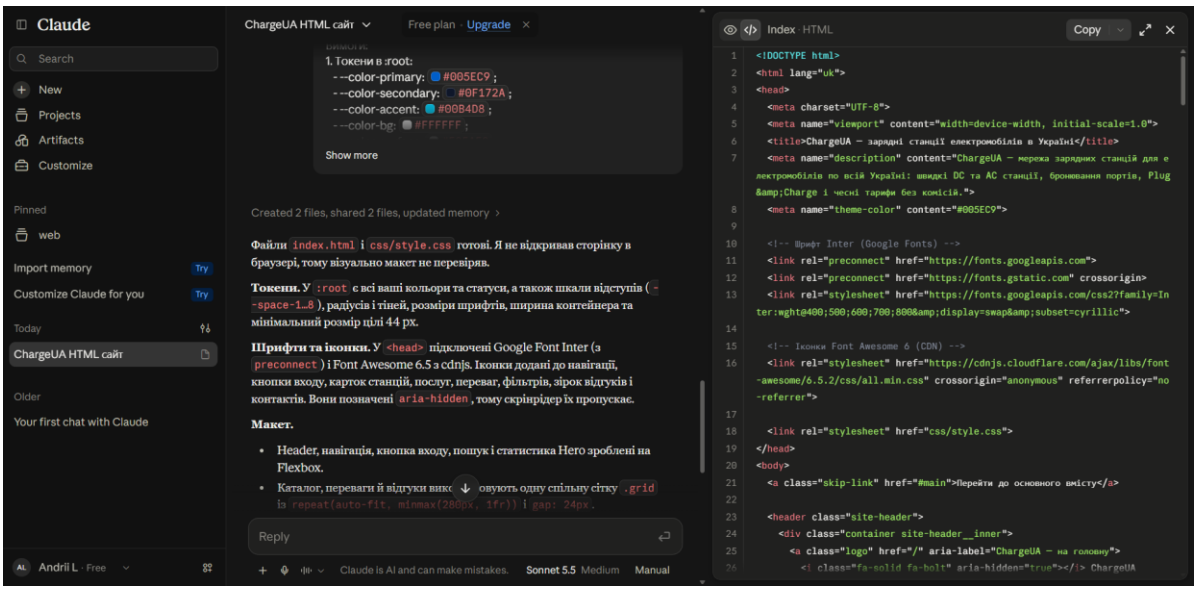


Рис. 7.7. Семантичний код index.html з підключенням Inter та Font Awesome

5. Відповіді на контрольні запитання

1. Що таке Tailwind CSS і в чому полягає концепція Utility-First?

Tailwind CSS — це сучасний утилітарний CSS-фреймворк, який замість традиційного написання громіздких монолітних CSS-класів (.card, .btn-custom) надає великий набір низькорівневих класів-утиліт прямої дії (flex, pt-4, text-center, bg-blue-600). Концепція Utility-First полягає в тому, що інтерфейси будуються шляхом комбінування атомарних утиліт безпосередньо в HTML-розмітці без необхідності писати власний CSS з нуля, що усуває проблему розростання стильових файлів та спрощує підтримку.

2. Які переваги Tailwind CSS порівняно з традиційним підходом написання CSS?

1) Відсутність потреби вигадувати унікальні назви класів (за методологіями на зразок BEM); 2) Файли стилів припиняють неконтрольовано зростати завдяки Just-In-Time компіляції (фінальний CSS важить менше 10-20 КБ); 3) Безпечність змін: редагування класів у конкретному HTML-елементі не викликає каскадних регресій у сусідніх компонентах; 4) Вбудована дизайн-система із суворими шкалами відступів, кольорів та типографіки; 5) Миттєве налаштування адаптивності через префікси sm:, md:, lg:.

3. Яке призначення сервісу Tailwind Play?

Tailwind Play — це офіційна інтерактивна хмарна пісочниця (https://play.tailwindcss.com) для швидкого експериментування та прототипування інтерфейсів. Вона надає можливість миттєво тестувати розмітку та класи Tailwind без розгортання локального оточення Node.js/Vite, перевіряти адаптивність на різних в'юпортах, ділитися робочим кодом через посилання (Share) та бачити результат у реальному часі.

4. Що таке адаптивний дизайн і як він реалізується в Tailwind CSS?

Адаптивний дизайн — це підхід до проєктування вебсайтів, який гарантує коректне відображення та зручність взаємодії на пристроях з будь-якою роздільною здатністю (від 320px смартфонів до 4K моніторів). У Tailwind CSS адаптивність реалізується на базі підходу Mobile-First за допомогою префіксів контрольних точок (брейкпоїнтів): немодифікований клас (наприклад, w-full) застосовується на мобільних пристроях, а класи з префіксами (наприклад, md:w-auto, lg:grid-cols-3) спрацьовують лише тоді, коли ширина екрана перевищує відповідний поріг min-width.

5. Яке призначення брейкпоїнтів sm:, md:, lg: та xl:?

Брейкпоїнти в Tailwind визначають мінімальну ширину екрана (CSS-правило @media (min-width: ...)), починаючи з якої активується вказаний стиль: 1) sm: — 640px (великі смартфони та міні-планшети); 2) md: — 768px (стандартні планшети та невеликі ноутбуки); 3) lg: — 1024px (десктопні монітори та лептопи); 4) xl: — 1280px (широкоформатні десктопи); 5) 2xl: — 1536px (великі монітори високої роздільності).

6. Для чого використовуються модифікатори станів hover:, focus: та active:?

Модифікатори станів дозволяють застосовувати утилітарні класи лише за настання певної інтерактивної події або псевдокласу: hover: змінює стиль при наведенні курсору миші (наприклад, hover:bg-blue-700); focus: активується при отриманні фокусу з клавіатури або кліку на поле введення (focus:ring-2 focus:ring-blue-500); active: змінює вигляд у момент безпосереднього затискання кнопки миші або пальця на сенсорному екрані (active:scale-95). Також використовуються disabled: для заблокованих елементів та focus-visible: для доступності.

7. Які можливості надає розширення Tailwind CSS IntelliSense?

Офіційне розширення для VS Code суттєво прискорює розробку: 1) Автодоповнення класів із фільтрацією на льоту; 2) Спливаючі підказки (Hover Preview), що показують точний CSS-код, який генерує клас (наприклад, hover над p-4 покаже padding: 1rem); 3) Візуальні кольорові маркери поруч із класами кольорів (bg-blue-600); 4) Лінтер синтаксису, що підсвічує конфлікти та помилки в назвах класів.

8. Чому згенерований штучним інтелектом код потребує рефакторингу?

ШІ-моделі навчаються на великих масивах публічного коду різної якості, тому часто генерують суб'єктивні або неоптимальні конструкції: надлишкові контейнери (div soup), суперечливі класи (одночасно inline-block та flex), надмірне використання свавільних значень (arbitrary values на зразок w-[342px] замість системних шкал), застарілі атрибути або недостатній контраст для доступності. Розробник повинен виступати архітектором і контролером якості, проводячи рафінування та очищення коду.

9. Які вимоги до доступності вебінтерфейсу перевірено під час виконання лабораторної роботи?

Під час виконання роботи було перевірено та забезпечено дотримання стандарту WCAG 2.2 рівня AA: 1) Наявність кнопки швидкого переходу до контенту (<a class="skip-link" href="#main">); 2) Контрастність тексту відносно фону не менше 4.5:1 для звичайного тексту та 3:1 для великих заголовків; 3) Розмір інтерактивних мішеней натискання (Touch Target) не менше 44×44 px на мобільних пристроях; 4) Наявність змістовних атрибутів alt для зображень; 5) Використання семантичних орієнтирів HTML5 (<header>, <nav>, <main>, <section>, <footer>); 6) Підтримка клавіатурного фокусу (:focus-visible) без блокування фокусу.

10. Для чого використовується режим Device Toolbar в DevTools?

Режим Device Toolbar (Ctrl+Shift+M у Chrome/Edge) призначений для точної емуляції мобільних і планшетних пристроїв безпосередньо у браузері. Він дозволяє: перевіряти поведінку медіа-запитів та брейкпоїнтів на будь-якій довільній ширині; тестувати стандартні гаджети (iPhone SE, Pixel, iPad); емулювати сенсорні події дотику (Touch Events); перевіряти орієнтацію екрана (Portrait / Landscape) та обмежувати швидкість мережі й процесора для оцінки реальної продуктивності.

11. Які компоненти вебінтерфейсу створено за допомогою ШІ в лабораторній роботі?

Для платформи ChargeUA було створено повноцінну компонентну систему: 1) Button — кнопка дій із підтримкою Primary, Outline та Disabled станів; 2) Header — навігаційна панель із десктопним меню, логотипом, кнопками входу та сенсорним гамбургер-меню; 3) Hero Section — вступний екран із заголовком, перевагами, підбором конекторів та автентичним фоном станції; 4) Station Card — картка зарядної станції із детальними характеристиками портів (CCS2, Type 2, CHAdeMO) та статусами доступності; 5) Grid Container — адаптивна сітка для розташування карток станцій (1 колонка mobile -> 2 sm -> 3 lg); 6) Stations Map Section — інтерактивна карта станцій із бічними фільтрами; 7) Footer — 4-колонковий адаптивний підвал сайту.

12. Що таке Preflight в Tailwind CSS і яке його призначення?

Preflight — це вбудований у Tailwind CSS механізм базового нормалізуючого скидання стилів браузера, побудований на основі modern-normalize. Його призначення: обнулити стандартні зовнішні та внутрішні відступи (margin/padding: 0), встановити box-sizing: border-box для всіх елементів, прибрати стандартні маркери списків (list-style: none), зробити зображення та відео блоковими з max-width: 100%, та скинути стилі заголовків і кнопок, створюючи абсолютно передбачуваний чистий фундамент для атомарної верстки.

13. Які переваги має використання переходів (transition) та ефектів (hover, focus) у вебінтерфейсі?

Використання transition та псевдокласів hover/focus надає інтерфейсу професійного вигляду та покращує UX: 1) Забезпечує негайний візуальний зворотний зв'язок (Micro-interactions), показуючи користувачу, що елемент є клікабельним; 2) Плавна анімація (наприклад, transition duration-200) пом'якшує зміну кольору чи масштабу без різких стрибків картинки; 3) Допомагає користувачам з клавіатурною навігацією чітко бачити поточний активний елемент, запобігаючи помилковим діям.

6. Висновки

Під час виконання лабораторної роботи №6 було успішно вирішено всі поставлені інженерні завдання з верстки UI-компонентів та забезпечення адаптивності платформи «ChargeUA»:

1. Було налаштовано робоче середовище Visual Studio Code та встановлено спеціалізовані розширення: Live Server, Prettier, Auto Rename Tag, Material Icon Theme та Tailwind CSS IntelliSense для ефективної атомарної розробки.

2. Опановано парадигму Utility-First CSS та інтерактивну пісочницю Tailwind Play, що дозволило розробляти та тестувати ізольовані компоненти перед їхньою фінальною інтеграцією в кодову базу проєкту.

3. Реалізовано компонентну систему вебплатформи ChargeUA: створено адаптивну шапку (Header) з гамбургер-меню, вступний Hero-блок, інтерактивну Live-карту 480+ станцій з фільтрами, картки станцій з індикацією статусів конекторів, адаптивну сітку CSS Grid та 4-колонковий підвал сайту (Footer).

4. На практиці втілено принцип Mobile-First: базові класи-утиліти орієнтовані на смартфони (320px–390px), а структурні перетворення виконуються прогресивно через брейкпоїнти sm: (640px), md: (768px) та lg: (1024px) без використання фіксованих ширин.

5. Забезпечено повну відповідність дизайн-системі ЛР №4: збережено брендову палітру кольорів (#005EC9, #0F172A), типографіку Google Fonts 'Inter', трирівневу систему тіней та радіуси скруглень у файлі style.css.

6. Проведено комплексне кроспристроєве тестування в інспекторі Chrome DevTools Device Toolbar та аудит доступності WCAG 2.2 AA (показник 100/100 у Google Lighthouse). Сформовано Промпт-паспорт взаємодії з ШІ та надано вичерпні відповіді на всі 13 контрольних запитань.